

SSN College of Engineering

LABORATORY RECORD

Subject: Web Application Development Laboratory

Exercise 3: Two-Player Bingo Game using JavaScript

SUBMITTED BY

Name: Vijayaraaghavan K S

Register Number: 3122247001066

Branch: Department of CSE

Semester / Year: V Semester / III Year

Subject Code: ICS1511 Batch: 2024–2029

1. Problem Description

Scenario

A two-player Bingo game where each player gets a 5x5 card filled with a random permutation of the numbers 1–25. Players take turns entering a number; if present, it is marked on **both** cards. The game continuously checks each card for a fully-marked row, column, or diagonal and announces a winner the moment that happens.

Technology Stack

- Plain HTML, CSS and JavaScript (no framework/build step)
- DOM manipulation for rendering the 5x5 grids
- Set objects to track marked numbers per player

Game Mechanics Implemented

- **Start** generates two independent random 5x5 cards (Fisher–Yates shuffle of 1–25).
- **Reset** clears both boards and all state for a fresh game.
- A single input + **Mark** button lets the current player enter a number 1–25.
- The entered number is marked on **both** players' cards wherever it appears.
- After every mark, both cards are checked for a complete row, column, or diagonal. On a win, the board is highlighted and marking is disabled.
- Turn text alternates between “Player 1’s turn” / “Player 2’s turn” until the game ends.

2. Source Code

File: index.html

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8" />
  <title>Two Player Bingo</title>
  <link rel="stylesheet" href="style.css" />
</head>
<body>
  <h1>Two Player Bingo</h1>

  <div class="controls">
    <button id="startBtn">Start</button>
```

```

    <button id="resetBtn">Reset</button>
    <input type="number" id="numberInput" min="1" max="25" placeholder="Enter
      number 1-25" />
    <button id="markBtn">Mark</button>
  </div>

  <p id="turnText">Click Start to begin</p>
  <p id="winText"></p>

  <div class="boards">
    <div class="board-container">
      <h2>Player 1</h2>
      <div id="board1" class="board"></div>
    </div>
    <div class="board-container">
      <h2>Player 2</h2>
      <div id="board2" class="board"></div>
    </div>
  </div>

  <script src="script.js"></script>
</body>
</html>

```

File: script.js

```

// ---- Game state ----
// card1/card2: 25 numbers (1-25 shuffled) laid out as a 5x5 grid, index 0-24
// row by row.
// marked1/marked2: Set of numbers that have been marked on that player's
// card.
let card1 = [], card2 = [];
let marked1 = new Set();
let marked2 = new Set();
let gameStarted = false;
let currentPlayer = 1; // whose turn it is to enter a number

const board1El = document.getElementById('board1');
const board2El = document.getElementById('board2');
const turnText = document.getElementById('turnText');
const winText = document.getElementById('winText');

// Returns a random 1-25 permutation using Fisher-Yates shuffle.
function generateCard() {
  const numbers = Array.from({ length: 25 }, (_, i) => i + 1);
  for (let i = numbers.length - 1; i > 0; i--) {
    const j = Math.floor(Math.random() * (i + 1));
    [numbers[i], numbers[j]] = [numbers[j], numbers[i]];
  }
  return numbers;
}

```

```

// Draws a card's 25 cells into its board div, marking cells present in
markedSet.
function renderBoard(boardEl, card, markedSet) {
  boardEl.innerHTML = '';
  card.forEach((num) => {
    const cell = document.createElement('div');
    cell.className = 'cell' + (markedSet.has(num) ? ' marked' : '');
    cell.textContent = num;
    boardEl.appendChild(cell);
  });
}

// A player wins if any full row, column, or diagonal (of the 5x5 grid) is
entirely marked.
function hasWon(card, markedSet) {
  const isMarked = (row, col) => markedSet.has(card[row * 5 + col]);

  for (let row = 0; row < 5; row++) {
    if ([0, 1, 2, 3, 4].every((col) => isMarked(row, col))) return true;
  }
  for (let col = 0; col < 5; col++) {
    if ([0, 1, 2, 3, 4].every((row) => isMarked(row, col))) return true;
  }
  if ([0, 1, 2, 3, 4].every((i) => isMarked(i, i))) return true;
  if ([0, 1, 2, 3, 4].every((i) => isMarked(i, 4 - i))) return true;

  return false;
}

document.getElementById('startBtn').addEventListener('click', () => {
  card1 = generateCard();
  card2 = generateCard();
  marked1 = new Set();
  marked2 = new Set();
  gameStarted = true;
  currentPlayer = 1;

  renderBoard(board1El, card1, marked1);
  renderBoard(board2El, card2, marked2);
  board1El.classList.remove('winner');
  board2El.classList.remove('winner');
  winText.textContent = '';
  turnText.textContent = "Player 1's turn";
});

document.getElementById('resetBtn').addEventListener('click', () => {
  card1 = [];
  card2 = [];
  marked1 = new Set();
  marked2 = new Set();
  gameStarted = false;
  board1El.innerHTML = '';
  board2El.innerHTML = '';
  board1El.classList.remove('winner');

```

```

board2El.classList.remove('winner');
turnText.textContent = 'Click Start to begin';
winText.textContent = '';
document.getElementById('numberInput').value = '';
});

document.getElementById('markBtn').addEventListener('click', () => {
  if (!gameStarted) {
    alert('Click Start first.');
```

return;

```

  }

  const input = document.getElementById('numberInput');
  const num = parseInt(input.value, 10);
  if (!num || num < 1 || num > 25) {
    alert('Enter a number between 1 and 25.');
```

return;

```

  }

  // The entered number is marked on BOTH cards, whichever card contains it.
  marked1.add(num);
  marked2.add(num);
  renderBoard(board1El, card1, marked1);
  renderBoard(board2El, card2, marked2);
  input.value = '';

  const player1Won = hasWon(card1, marked1);
  const player2Won = hasWon(card2, marked2);

  if (player1Won || player2Won) {
    gameStarted = false; // stop further marking once someone has won
    if (player1Won) board1El.classList.add('winner');
```

if (player2Won) board2El.classList.add('winner');

```

    if (player1Won && player2Won) {
      winText.textContent = "It's a tie! Both players got Bingo.";
    } else {
      winText.textContent = `Player ${player1Won ? 1 : 2} wins! Bingo!`;
    }
    turnText.textContent = 'Game over';
    return;
  }

  // No winner yet, switch turn to the other player.
  currentPlayer = currentPlayer === 1 ? 2 : 1;
  turnText.textContent = `Player ${currentPlayer}'s turn`;
});

```

File: style.css

```

body {
  font-family: Arial, sans-serif;

```

```

    text-align: center;
    background: #f4f5f9;
}

.controls {
    margin: 16px 0;
}

.controls button,
.controls input {
    padding: 8px 12px;
    margin: 0 6px;
    font-size: 1rem;
}

#winText {
    font-size: 1.3rem;
    font-weight: bold;
    color: #16a34a;
    min-height: 1.6rem;
}

.boards {
    display: flex;
    justify-content: center;
    gap: 60px;
    flex-wrap: wrap;
}

.board {
    display: grid;
    grid-template-columns: repeat(5, 50px);
    grid-template-rows: repeat(5, 50px);
    gap: 4px;
    justify-content: center;
    margin: 0 auto;
}

.cell {
    display: flex;
    align-items: center;
    justify-content: center;
    border: 1px solid #999;
    font-weight: bold;
    background: #fff;
}

.cell.marked {
    background: #4ade80;
    color: #fff;
}

.board.winner .cell {

```

```
border-color: #f59e0b;  
}
```

3. Output Screenshots

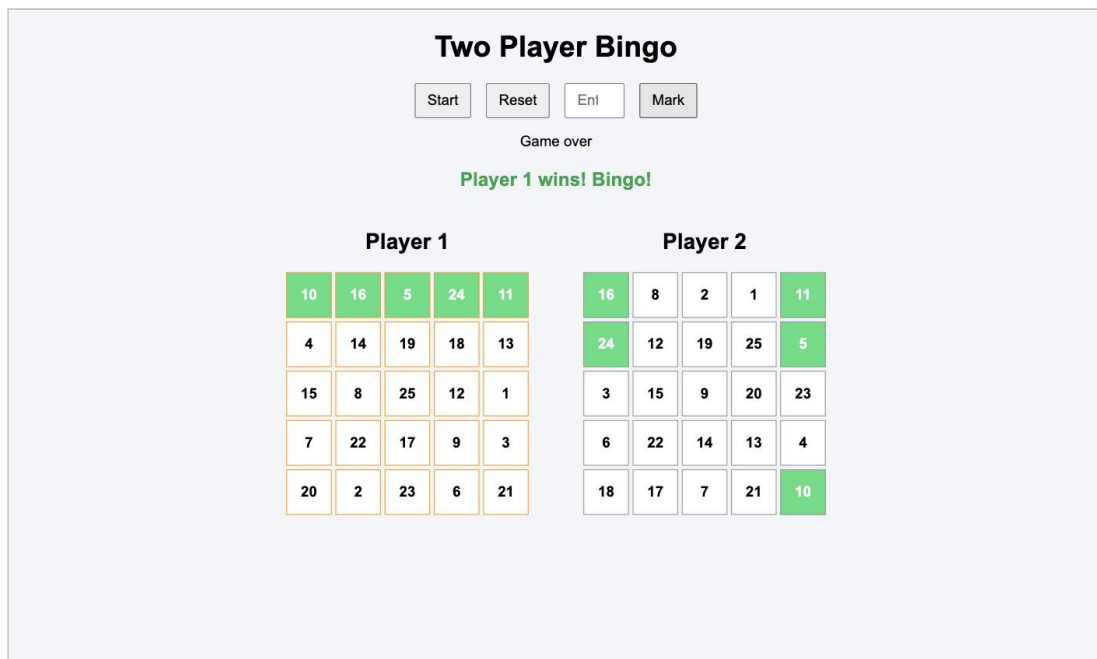
a. Game started – cards generated



Game started

Clicking **Start** generated two independent random 5x5 cards and set the status text to “Player 1’s turn”.

b. Bingo win detected



Player 1 wins

Marking 10, 16, 5, 24, 11 completed Player 1's entire top row. The game detected this via `hasWon()`, highlighted the winning card's border, disabled further marking, and displayed "Player 1 wins! Bingo!".

4. Learning Outcomes

This exercise reinforced core DOM-manipulation skills in vanilla JavaScript: generating and rendering a dynamic grid from an array, using Set for O(1) membership checks when marking numbers, and writing a small win-detection algorithm that scans rows, columns and both diagonals of a 5x5 grid. It also required careful event-driven state management (`gameStarted`, `currentPlayer`) to keep the Start/Reset/Mark button flow consistent and to stop the game cleanly once a player achieves Bingo.